

CS 305: THEORY OF COMPUTATION

Module 2: Context-Free Languages, Grammars & Modern Parsing

Weeks 6–8: From Compiler ASTs to LLM Structured Decoding

Department of Computer Science

School of Technology

Ghana Institute of Management and Public Administration (GIMPA)

Level 300 Undergraduate · Semester 1

Module 2 Roadmap: Theory ↔ Compilers & Modern AI

Theoretical Foundations (Sipser Ch. 2)

- **Week 6:** Context-Free Grammars (CFGs) & Derivations
- **Week 6:** Parse Trees, Ambiguity & Chomsky Normal Form (CNF)
- **Week 7:** Pushdown Automata (PDAs) & Stack Memory
- **Week 7:** Equivalence Theorem: $\mathcal{L}(\text{CFG}) = \mathcal{L}(\text{PDA})$
- **Week 8:** Pumping Lemma for CFLs & Non-CFL Proofs

Research Literature & System Spotlights

- **Week 6: NeurIPS 2023:** Grammar-Guided LLM JSON Generation (outlines)
- **Week 7: PLDI 2011:** Adaptive LL(*) Parser Generators (ANTLR v4)
- **Week 8: TACL 2020:** Attention Failures on Dyck Languages (Stanford)

Central Thesis: Regular languages describe flat tokens; context-free grammars govern recursive hierarchical syntax across compilers, JSON parsers, and generative AI.

Week 6: Context-Free Grammars (CFGs)

Formal 4-Tuple Definition (Sipser Def 2.2) A **Context-Free Grammar (CFG)** is a 4-tuple $G = (V, \Sigma, R, S)$:

1. V : A finite set of **variables** (also called non-terminals).
2. Σ : A finite set of **terminals**, disjoint from V ($V \cap \Sigma = \emptyset$).
3. R : A finite set of **rules** of the form $A \rightarrow u$, where $A \in V$ and $u \in (V \cup \Sigma)^*$.
4. $S \in V$: The designated **start variable**.

Canonical Example: Palindromes and Balanced Nesting Let $G_1 = (\{S\}, \{0, 1\}, R, S)$ with production rules:

$$S \rightarrow 0S0 \mid 1S1 \mid \varepsilon$$

Generates all even-length binary palindromes: $L(G_1) = \{ww^R \mid w \in \{0, 1\}^*\}$.

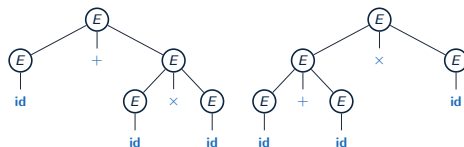
- **Derivation trace** for $w = 0110$: $S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 01\varepsilon10 = 0110$.
- **Regular Limit**: $L(G_1)$ is strictly non-regular (requires unbounded matching memory).

Week 6: Derivations, Parse Trees & Ambiguity

Formal Grammar Ambiguity (Sipser Def 2.7)

- A string w is derived **ambiguously** in CFG G if it has **two or more distinct parse trees** (or distinct leftmost derivations).
- **Inherent Ambiguity:** Some CFLs (e.g., $\{a^i b^j c^k \mid i = j \vee j = k\}$) have *no* unambiguous CFG.
- **Compiler Hazard:** Ambiguity causes undefined semantics in programming language execution!

Conflicting Trees: $E \rightarrow E + E \mid E \times E \mid \text{id}$



Tree A: $\text{id} + (\text{id} \times \text{id})$

Tree B: $(\text{id} + \text{id}) \times \text{id}$

Resolution: Enforce precedence tiers ($E \rightarrow E + T \mid T, T \rightarrow T \times F \mid F$).

Week 6: Chomsky Normal Form (CNF)

Definition & Canonical Structure (Sipser Def 2.8) A Context-Free Grammar is in **Chomsky Normal Form (CNF)** if every rule has one of three forms:

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a \quad \text{or} \quad S \rightarrow \varepsilon \quad (\text{where } B, C \in V \setminus \{S\} \text{ and } a \in \Sigma).$$

The 4-Step Conversion Algorithm

1. **New Start Symbol:** Add rule $S_0 \rightarrow S$ to prevent the start symbol from appearing on any RHS.
2. **Eliminate ε -Rules:** For rule $A \rightarrow \varepsilon$, remove it and replace every $R \rightarrow uAv$ with $R \rightarrow uv$.
3. **Eliminate Unit Rules:** For rule $A \rightarrow B$, remove it and copy all productions $B \rightarrow u$ into $A \rightarrow u$.
4. **Binarize Long RHS:** Replace $A \rightarrow u_1 u_2 \dots u_k$ ($k \geq 3$) with cascade $A \rightarrow u_1 C_1, C_1 \rightarrow u_2 C_2, \dots$

Why CNF Matters for Algorithms

- **Binary Derivation Bound:** Any string $w \in L(G)$ of length $n = |w| \geq 1$ requires **exactly $2n - 1$ steps** to derive!
- **Decidability:** Instantly proves CFL membership is decidable (finite search space).
- **CYK Algorithm:** Enables dynamic programming parsing in $\mathcal{O}(n^3)$ polynomial time.

Week 6 Spotlight: Grammar-Guided LLM Structured Decoding

Seminal Literature (NeurIPS Foundation Models 2023): Willard & Louf, "Efficient Guided Generation for Large Language Models," NeurIPS 2023 (HuggingFace outlines).

The Industrial Production Problem

- **Malformed JSON / SQL:** Modern LLMs produce syntax errors (missing closing braces, hallucinated keys) in 5–15% of API calls.
- **Brittle Retries:** Re-prompting adds latency ($\geq 800\text{ms}$) and enterprise billing cost without safety guarantees.

The Automata Solution: PDA Logit Masking

- **Pushdown Grammar Parser:** Compile JSON Schema into an equivalent Pushdown Automaton / CFG parser.
- **Logit Masking:** Mask invalid tokens at decoding step t :

$$\text{Logit}_v = -\infty \quad \forall v \notin \text{NextTokens}(S_t)$$

- **Guarantee: 100% mathematically valid syntax** with zero inference retries!

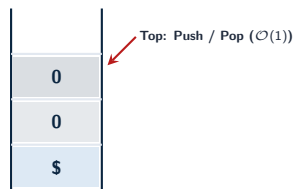
Week 7: Pushdown Automata (PDAs)

Formal 6-Tuple Definition (Sipser Def 2.13)

A **Nondeterministic Pushdown Automaton (PDA)** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$:

1. Q : Finite set of internal **states**.
2. Σ : Finite input **alphabet**.
3. Γ : Finite **stack alphabet** (storage symbols).
4. $\delta : Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \rightarrow \mathcal{P}(Q \times \Gamma_{\epsilon})$: Transition function.
5. $q_0 \in Q$: Designated **start state**.
6. $F \subseteq Q$: Set of **accept states**.

The LIFO Stack Memory Architecture

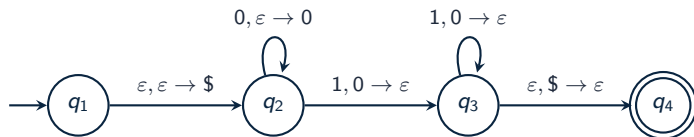


LIFO Stack Γ^*

Stack Limitation: Infinite capacity, but strictly restricted access. Deep elements cannot be read without popping top symbols!

Week 7: PDA State Diagram: Recognizer for $L = \{0^n 1^n \mid n \geq 0\}$

Transition Convention: $a, b \rightarrow c$ denotes: “Read input a , Pop b from stack, Push c onto stack”



Execution Trace on String $w = 0011$

Step	State	Input	Stack	Operation Description
0	q_1	0011	ϵ	Initial configuration
1	q_2	0011	$\$$	Push bottom marker $\$$
2	q_2	011	$0\$$	Read '0', push '0'
3	q_2	11	$00\$$	Read '0', push '0'
4	q_3	1	$0\$$	Read '1', pop matched '0'
5	q_3	ϵ	$\$$	Read '1', pop matched '0'
6	q_4	ϵ	ϵ	Stack empty; accept state $\implies$ ACCEPTED!

Week 7: Fundamental Equivalence: $\mathcal{L}(\text{CFG}) = \mathcal{L}(\text{PDA})$

Fundamental Theorem of Context-Free Languages (Sipser Thm 2.20) A language L is context-free if and only if some pushdown automaton recognizes it:

$$\mathcal{L}(\text{CFG}) \equiv \mathcal{L}(\text{PDA}) = \text{Context-Free Languages.}$$

Direction 1: $\text{CFG} \implies \text{PDA}$ (Sipser Lemma 2.21)

- **Top-Down Simulation:** The PDA simulates the leftmost derivation on its stack.
- If top of stack is variable A , nondeterministically replace it with RHS of rule $A \rightarrow u$.
- If top of stack is terminal a , match against incoming input symbol and pop.

Direction 2: $\text{PDA} \implies \text{CFG}$ (Sipser Lemma 2.27)

- **State Pair Variables:** For every pair of states (p, q) , create variable A_{pq} generating all strings driving PDA from p to q with net empty stack.
- Compose grammar rules capturing push-pop cycles: $A_{pq} \rightarrow aA_{rs}b$ and $A_{pq} \rightarrow A_{pr}A_{rq}$.

Compiler Runtime Mirror: Modern programming call stacks directly implement PDA push-pop semantics.

Week 7 Spotlight: Production Parser Generators (ANTLR v4)

Seminal Literature (ACM SIGPLAN PLDI 2011): Parr et al. (Univ. of San Francisco), *"Adaptive LL(*) Parsing: The Power of Dynamic Analysis,"* PLDI 2011.

Traditional Theory Limit: Static LL(k) / LR(k)

- **Fixed Lookahead:** Classical parsers enforce $k = 1$ token lookahead to guarantee deterministic $\mathcal{O}(n)$ parsing.
- **Grammar Refactoring Pain:** Forces software engineers to manually left-factor and eliminate left recursion from their grammars.

The Systems Solution: Adaptive LL(*)

- **Dynamic DFA Lookahead:** ANTLR launches a sub-DFA at runtime to read arbitrary token lookahead only when ambiguity is detected!
- **Industrial Scale:** Powers **SQLite**, **PostgreSQL** query engines, **Java / Python** compilers, and IDE syntax highlighters.

Week 8: The Pumping Lemma for Context-Free Languages

Formal Theorem Statement (Sipser Thm 2.34) If L is a Context-Free Language, then there exists a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be divided into $s = uvxyz$ satisfying three conditions:

1. $uv^i xy^i z \in L$ for each $i \geq 0$ (**Simultaneous Double Pumping Property**),
2. $|vy| > 0$ (**At least one of v or y is non-empty**),
3. $|vxy| \leq p$ (**Pumping Window Bounded by p**).

Parse Tree Geometric Pigeonhole Principle

- In a CNF grammar, a parse tree of height h yields $|s| \leq 2^{h-1}$.
- If $|s| \geq p = 2^{|V|+1}$, the tree height must exceed $|V| + 1$.
- Along the longest path, **at least one variable $R \in V$ must repeat** by the Pigeonhole Principle!

Subtree Replacement Mechanics

- The upper occurrence of R derives vRy .
- The lower occurrence of R derives x .
- We can substitute the lower tree with the upper tree ($i \geq 2$) or delete the loop ($i = 0$), producing valid strings $uv^i xy^i z \in L$.

Week 8: Adversarial Proof Template: Non-CFLs

Adversarial Proof Template: Showing $L = \{a^n b^n c^n \mid n \geq 0\}$ is Non-Context-Free

1. **Proof by Contradiction:** Assume L is context-free $\implies$ Pumping Lemma guarantees pumping length p .
2. **Select String:** Choose $s = a^p b^p c^p \in L$. Note that $|s| = 3p \geq p$.
3. **Apply Window Bound ($|vxy| \leq p$):** Substring vxy can span at most **two distinct symbol types** (either $\{a, b\}$ or $\{b, c\}$, never all three $\{a, b, c\}$).
4. **Pump Down ($i = 0$):** In string $s' = uv^0xy^0z = uxz$, the count of at most two symbols decreases, while the third remains untouched at p .
5. **Contradiction:** Symbol counts are no longer equal $\implies s' \notin L \implies L$ is **strictly non-context-free!**

Non-CFL Case 2: Exact Duplication $L = \{ww \mid w \in \{0, 1\}^*\}$. Requires non-LIFO cross-serial matching; a single stack cannot compare symbols without inverting sequence order.

Week 8 Spotlight: Self-Attention Limits on Dyck Languages

Seminal Literature (Transactions of the ACL 2020): Michael Hahn (Stanford University), “*Theoretical Limitations of Self-Attention in Neural Sequence Models*,” TACL 2020.

The Dyck- k Hierarchical Benchmark

- **Nested Brackets:** Dyck- k consists of properly balanced parentheses: $[(\{ \})]$.
- **Trivial for PDAs:** Solvable in $\mathcal{O}(n)$ linear time with a simple stack push/pop counter.

Why Standard Transformers Fail

- **Circuit Depth Bound (TC^0):** Self-attention with fixed depth lacks recursive state tracking.
- **Theoretical Proof:** Hahn proves that self-attention cannot evaluate hierarchical nesting depth beyond its context window without exponential precision!

Module 2 Summary & Looking Ahead to Module 3

Module 2 Key Scientific Takeaways

- **The Power of Stack Memory:** Pushdown Automata and CFGs elevate computation from flat regular tokens to recursive nested trees (ASTs, JSON).
- **The Boundary of Stack Memory:** A single LIFO stack cannot verify multi-way equality ($a^n b^n c^n$) or cross-serial dependencies (ww).

Looking Ahead: Module 3 (Weeks 9–12)

- **The Tape Upgrade:** What happens if we replace the restricted LIFO stack with an **infinite, bidirectional read/write tape**?
- **Turing Completeness:** We enter the realm of unrestricted computation: **Turing Machines**, the **Church–Turing Thesis**, and the unbreakable boundary of the **Halting Problem**!

Midterm Exam Preparation: Comprehensive review of Modules 1 & 2 in Week 8.